

Phase 1: Assisted Development

Module 2: Context Development

Building and managing a context framework

Agenda

- Learning objectives
- Where we are in the journey
- Your codebase has two layers
- Types of context
- Context Framework Contents
- Configuring the entry point
- Three Context Tips
- Four Collaboration Tips
- Prior Art: Context Frameworks
- AI Tool Landscape
- The evolution of our Config Service

Learning Objectives

- **Goal:** Build a usable context framework for a web API and its UI
- **Takeaway:** The beginnings of a context framework for future use
- **Hope:** You will use this context framework in the future

Along the way you will:

1. **Collaboratively** craft context documents with your assistant
2. **Take control** of when context is brought into short-term memory
3. **Compare its value** and cost with not using structured context

Where We Are

Module 2 of 9 — we're building the foundations.

Phase	Modules	Focus
Assisted	1–3	Foundations: context, workflow, collaboration
Pairing	4–6	Skills, tools, your pair agent
Agentic	7–9	Agents that work alongside you

Module 1 gave you the collaboration mindset. Module 2 gives your assistant something to work with — a persistent context it can rely on across sessions.

By Module 6 you'll have a **pair agent** — and the context framework you build here is what that agent will use to know your project.

Your codebase has two layers

- **Code** — the implementation, tests, and configuration
- **Context** — structured knowledge that surrounds it: plans, constraints, decisions

A prompt is ephemeral. Context lives in the repo, survives sessions, and works across tools.

Sean Grove, OpenAI: "*A written specification aligns humans*" — and it aligns your AI assistant too.

Managing both layers is the shift this module is about.

Types of Context

Module 2 — what we're building now:

- **Working context** — the active conversation and scratch files specific to current work item
- **Semantic context** — persistent "what" knowledge. Lives in the repo, survives sessions, works across tools.

Module 3 — what we'll add next:

- **Episodic context** — when and why: work items, in-progress changes, retrospectives
- **Procedural context** — how: scripts, Makefile, runbooks, skills

This is *context* for our assistants and *memory* and *knowledge* for our agents

Context Framework Contents

Current focus:

- **About:** Project description, problem it's solving, user personas, constraints
- **Architecture:** Patterns, module relationships, data flow, persistence
- **Implementation:** Languages, versions, external dependencies, preferences
- **Navigation:** A README in each major folder — your project's knowledge graph

Next, in Module 3:

- **Planning:** high-level plan, current status, task breakdown
- **Tooling:** environment details, CLI tools

Configuring the entry point

- All AI coding tools have a mechanism to auto-load context
 - Often called "rules" — Cursor, Cline, Claude Code, Codex all support it
- `AGENTS.md` has emerged as the cross-tool standard
 - Cline, Claude Code, and most others recognise it automatically
- Use it to reference your context files — e.g. `memory/ABOUT.md`
- Load files selectively as needed to manage context window cost
- Progressive disclosure uses short descriptions and links between files

Prior Art: Context Frameworks

Others have recognised this problem formally:

- [memory-bank](#) from Cline
- [AgentOS](#) from Brian Casel
- [Spec Kit](#) from GitHub
- [BMad Method](#) from Brian Madison

The variety confirms the problem is real. We're building our own to develop our intuition for what works.

Context Tip #1

Clean context is happy context

Clean the window when it contains:

- **Incorrect** information
- **Missing** important details
- **Noisey** or irrelevant details

Context Tip #2

Interesting combination of syntax

```
<conditional-block context-check="general-formatting">  
IF this General Formatting section already read in current context:  
    SKIP: Re-reading this section  
    NOTE: "Using General Formatting rules already in context"  
ELSE:  
    READ: The following formatting rules  
  
## General Formatting  
  
...
```

Your assistant will know what you mean.

Context Tip #3

Your CFX will be as familiar as what you're building

- If you're building something for the first time, expect to start your context framework from [mostly] scratch - and that's okay
- If you already have existing assets (docs, code, etc), use that to collaboratively build your context framework
- Expect to iterate on it more than you expect 😊

Collaboration Tip #1

Scope + Context = Precision

Be specific about

- **What** you want the AI to do (scope)
- **With** the assets you provide (context)
- **Which files** the assistant should read and work from

Example:

"Using `memory/ARCHITECTURE.md` and `memory/IMPLEMENTATION.md`, refactor the EventHandler class in `@events/EventHandler.ts` to handle its events asynchronously using the queue."

Collaboration Tip #2

Don't assume you're right, validate your assumptions

- Use your assistant as a skeptical critical thinker
- Your context docs are assumptions too — ask the assistant to challenge them
- Leave yourself 20% of uncertainty as space to be surprised
- Role: skeptic

Example:

"What are 3 things missing or potentially wrong in `memory/ARCHITECTURE.md` that could mislead someone building on this service?"

Collaboration Tip #3

Plan before you act

- Greatly increases precision (across *all* models)
- Reduces the difference between what you expected and what you got
- Great at creating examples when you don't already have them
- Role: planner

Example:

"Without making any filesystem changes, please describe the steps you would take to fix all of the type checking errors and warnings."

Collaboration Tip #4

Ask your assistant to explain themselves

- You *MUST* read and understand all of YOUR code
- So if you see something you don't understand ... just ask
- These prompts are particularly useful when using unfamiliar languages & frameworks
- Role: mentor

Example:

"Please explain why you used the `Protocol` type in `@socket_handler_interface.py` instead of an ABC? And what does `...` mean?"

AI Tool Landscape

Categories

- **IDE integrations** (Cursor, Cline, RooCode, Windsurf)
- **Standalone assistants** (Claude Code, Codex, OpenCode, Antigravity)
- **Specialized tools** (pipeline integrations, PR tools)

Selection Criteria

- Cost
- Integration requirements
- Precision, and accessibility
- Model options per feature (BYO?)
- The coding to agent coordination spectrum

Project Progression: Module 2

The Config API Service — adding context

- A context framework for the existing Config Service
- `ABOUT.md` , `ARCHITECTURE.md` , `IMPLEMENTATION.md`
- An `AGENTS.md` entry point that auto-loads them
- Practice evaluating AI output — when to accept, revise, or revert
- Planning a web client library using your new context
- A `README.md` in each major folder — the start of a knowledge graph

Let's Begin

1. Start with `memory/ABOUT.md`
2. Create `AGENTS.md` to auto-load your context files
3. Test your assistant's understanding